

系統測試導向開發計畫與實施報告

資管三乙 第二組

目錄

一、GPT 開發計畫

1. 專案目標

2. 系統架構設計與各子系統開發計畫

2.1 架構概述

2.2 模組劃分

(1) 違規處理子系統開發計畫

功能描述

開發計畫

技術實現

(2) 車籍驗證子系統開發計畫

功能描述

開發計畫

技術實現

(3) AI 辨識子系統開發計畫

功能描述

開發計畫

技術實現

(4) 人工辨識子系統開發計畫

功能描述

開發計畫

技術實現

(5) 罰單生成子系統開發計畫

功能描述

開發計畫

技術實現

3. AI 開發計畫與注意事項

3.1 使用 AI 的開發流程

二、各單元之測試腳本

1. 違規處理子系統測試腳本

1.1 測試目標

1.2 測試項目

1.3 測試案例

案例 1：事件資料接收與分流測試

1.4 測試範圍

1.5 測試環境

1.6 測試總結

2. 車籍驗證子系統測試腳本

2.1 測試目標

2.2 測試項目

2.3 測試案例

案例 1：車牌合法性驗證與異常資料處理測試

案例 2：資料儲存測試

2.4 測試範圍

2.5 測試環境

2.6 測試總結

3. AI 辨識子系統測試腳本

1. 測試目標

2. 測試項目

3. 測試案例

案例 1：影像數據處理測試

案例 2：車牌號碼辨識測試

案例 3：辨識失敗處理測試

案例 4：資料儲存測試

案例 5：錯誤碼機制測試

4. 測試範圍

5. 測試環境

6. 測試總結

4. 人工辨識子系統測試腳本

1. 測試目標

2. 測試項目

3. 測試案例

案例 1：數據接收與人工審核測試

案例 2：數據更新與異常處理測試

案例 3：審核歷程記錄測試

4. 測試範圍

5. 測試環境

6. 測試總結

5. 罰單生成子系統測試腳本

1. 測試目標

2. 測試項目

3. 測試案例

案例 1：罰單生成與通知測試

案例 2：數據儲存與異常處理測試

案例 3：處理歷程記錄測試

4. 測試範圍

5. 測試環境

6. 測試總結

6. 全國違規事件管理儀表板子系統測試腳本

1. 測試目標

[2. 測試項目](#)

[3. 測試案例](#)

[案例 1：視覺化報表生成測試](#)

[4. 測試總結](#)

[三、開發測試過程](#)

[階段一：準備與環境搭建](#)

[階段二：程式碼結構調整](#)

[階段三：基本「違規處理子系統」框架搭建與功能開發](#)

[階段四：AI辨識、車籍確認、人工辨識子系統，切換至Tabnine使用](#)

[階段五：罰單生成子系統與切換至Vscode Copilot](#)

[總結：人工介入的重要性](#)

[四、檢討分析報告](#)

[個人能力檢討](#)

[團隊合作檢討](#)

[改進策略](#)

一、GPT 開發計畫

1. 專案目標

- 建立高效穩定的違規超速處理系統，實現從數據接收到處理、分類、通知的全流程自動化。
- 引入 AI 技術輔助開發，包括程式碼生成、註解補充、錯誤分析等，以提升開發效率並降低錯誤率。
- 確保系統具備良好的可維護性與擴展性，便於後續功能升級。

2. 系統架構設計與各子系統開發計畫

2.1 架構概述

- **前端**：基於 React.js，提供違規數據管理與測試介面。
- **後端**：使用 Node.js 和 Express.js 實現 API，負責數據接收、處理和存儲。
- **資料庫**：MySQL，用於存儲違規事件、車輛信息及處理記錄。
- **容器化**：使用 Docker 確保開發與生產環境一致性。
- **AI 工具**：
 - Tabnine 用於程式碼生成。
 - GitHub Copilot 用於功能補充和程式碼註解。

2.2 模組劃分

(1) 違規處理子系統開發計畫

功能描述

負責接收來自智慧燈桿的違規數據，完成分類處理，並分流至其他子系統。

開發計畫

1. 數據接收與格式化

- 實現 API 接口接收智慧燈桿數據（如車牌號碼、速度、地點）。
- 格式化接收到的數據，並存入 `EventBasicInfo` 表。
- 使用工具：Node.js（後端）。

2. 數據分流

- 將數據按車牌號碼傳遞至車籍驗證子系統。
- 傳遞影像數據至 AI 辨識子系統。
- 若數據不完整，記錄至異常日誌。

技術實現

- 前端：使用 JavaScript 與表單驗證檢查數據完整性。
 - 後端：Node.js 實現 API 邏輯。
-

(2) 車籍驗證子系統開發計畫

功能描述

驗證車牌號碼的合法性，並查詢車主相關信息。

開發計畫

1. 車牌驗證

- 整合資料庫 API，檢查車牌號碼是否有效。
- 使用假數據或模擬 API 進行測試。

2. 數據儲存

- 驗證結果存入 `VehicleInfo` 表，包括車主姓名、車型等資訊。

技術實現

- 後端：

- 實現數據查詢邏輯，若無效則返回錯誤狀態。
 - 使用 SQL 語句優化車牌檢索。
 - 測試工具：Postman 測試 API 請求。
-

(3) AI 辨識子系統開發計畫

功能描述

利用 Gemini API 對違規影像進行車牌號碼辨識。

開發計畫

1. 影像數據傳輸

- 設計 API 將影像數據傳送至 Gemini API，接收並格式化辨識結果。

2. 錯誤處理

- 對於辨識失敗的影像（如模糊），記錄錯誤原因並標記為「待審核」。

3. 數據儲存

- 成功辨識的結果存入 `AIRecognition` 表。

技術實現

- 與 Gemini API 進行數據交互。
 - 設計異步函數處理 API 響應，提高辨識效率。
-

(4) 人工辨識子系統開發計畫

功能描述

針對 AI 無法辨識的案件進行人工審核，並記錄處理歷程。

開發計畫

1. 案件接收與顯示

- 從 `ProcessingLog` 提取待審核案件，顯示於介面上，支持影像查看與數據填寫。

2. 審核與數據提交

- 人工輸入車牌號碼、車型和顏色，更新審核結果至 `ProcessingLog` 和 `EventBasicInfo`。

技術實現

- 前端：
 - 使用 React.js 開發互動界面，支持影像預覽（如 `<img>` 標籤）。
 - 後端：
 - 設計更新接口，將人工輸入的數據寫入資料庫。
-

(5) 罰單生成子系統開發計畫

功能描述

根據確認的違規案件生成罰單，並通知車主。

開發計畫

1. 罰單生成

- 提取確認的違規案件數據，生成罰單編號與罰鍰金額。
- 將罰單存入 `TicketInfo` 表。

2. 車主通知

- 更新通知狀態至 `TicketInfo` 表。

技術實現

- 前端：
 - 增加罰單預覽與列印功能。
 - 後端：
 - 設計接口生成罰單並更新通知狀態。
-

3. AI 開發計畫與注意事項

3.1 使用 AI 的開發流程

1. 需求分析與設計階段

- 繪製 DFD（資料流程圖），明確模組交互關係。
- 定義功能需求並轉化為具體任務。

2. 程式碼撰寫階段

- 使用 Tabnine 撰寫核心邏輯程式碼並附加詳細註解。
- 使用 GitHub Copilot 為現有程式碼補充功能或註解，並且註解時不更改原始邏輯。

3. 測試與 Debug 階段

- 利用 AI 為code增加協助分析錯誤訊息，但需人工審核修正方案。例如：

```
try {
    processSpeedData(inputData)
    Log.d(TAG, "Speed data processed successfully")
} catch (e: Exception) {
    Log.e(TAG, "Error processing speed data", e)
    throw CustomException("Speed data processing failed:
    ${e.localizedMessage}")
}
```

4. 版本控制與記錄

- 善用 Git 進行版本控制，每次提交清楚記錄變更內容，並且完成一個功能之後，就提交上傳，以免後續更改使前一個功能不能實現。

二、各單元之測試腳本

1. 違規處理子系統測試腳本

1.1 測試目標

確保違規處理子系統能正確接收來自智慧燈桿的數據，完成分類處理，並將數據正確分流至其他子系統（如車牌驗證、AI 辨識、人工辨識等）。

1.2 測試項目

1. 事件資料接收與分流測試

1.3 測試案例

案例 1：事件資料接收與分流測試

- 測試目標：
 - 確認系統能正確接收智慧燈桿傳送的數據。
 - 確認數據能正確儲存於資料庫。
 - 確認數據能正確分流至車牌驗證與 AI 辨識子系統。
- 輸入條件：
 - 舉發 ID（內定增加值）：V001

- 違規影像：影像1.jpg
- **測試步驟：**
 1. 啟動違規處理子系統。
 2. 模擬智慧燈桿傳送上述數據。
 3. 檢查系統日誌，確認數據是否被正確接收。
 4. 查詢資料庫 **EventBasicInfo** 表，確認數據是否正確儲存。
 5. 檢查影像是否成功傳遞至 AI 辨識子系統。
 6. 檢查數據是否成功傳遞至車牌驗證子系統。
- **預期結果：**
 - 系統顯示接收到完整數據。
 - 顯示 AI 辨識之車牌號碼。
 - **EventBasicInfo** 表中新增一筆記錄，包含舉發 ID、車牌號碼與影像。
 - 車牌號碼正確傳遞至車牌驗證子系統，驗證結果回傳並存入 **VehicleInfo**。
 - 違規影像成功傳遞至 AI 辨識子系統，辨識結果記錄於 **AIRecognition**。
 - AI 辨識結果回傳至違規處理子系統介面，與車牌驗證結果比較無誤，顯示「無誤」訊息。

1.4 測試範圍

- 涵蓋的功能：
 - 數據接收、儲存與分流邏輯。
- 涉及的資料表：
 - **EventBasicInfo**：儲存事件基本數據。
 - **VehicleInfo**：儲存車輛驗證結果。
 - **AIRecognition**：儲存 AI 辨識結果。

1.5 測試環境

- 測試伺服器：本地端Express.js
- 資料庫：MySQL 8.0
- 模擬工具：Postman（模擬智慧燈桿數據傳送）

1.6 測試總結

透過以上測試案例，能驗證違規處理子系統的功能正確性與穩定性，確保數據流向準確無誤，並能支援其他子系統的後續處理。

2. 車籍驗證子系統測試腳本

2.1 測試目標

確保車籍驗證子系統能正確處理來自違規處理子系統的車牌號碼資料，查詢車輛登記信息，並返回車主相關信息，同時正確記錄驗證結果。

2.2 測試項目

1. 車牌合法性驗證測試
2. 異常資料處理測試
3. 資料儲存測試

2.3 測試案例

案例 1：車牌合法性驗證與異常資料處理測試

- 測試目標：
 - 確認系統能正確驗證車牌是否存在於車籍資料庫中。
 - 確認在資料庫中不存在或格式異常的車牌時，能正確處理並記錄失敗原因。
- 輸入條件：
 - 車牌號碼：
 - ABC-1234（合法且存在於資料庫）
 - XYZ-9999（不存在於資料庫）
 - A12B34C（格式不符）
- 測試步驟：
 1. 傳入 **ABC-1234**，檢查是否返回正確的車主信息。
 2. 傳入 **XYZ-9999**，檢查是否回傳錯誤信息「車牌不存在」。
 3. 傳入 **A12B34C**，檢查是否回傳錯誤信息「車牌格式錯誤」。
- 預期結果：
 - 對於 **ABC-1234**：
 - 返回正確的車主信息（姓名：張三；地址：台北市中正區）。

- 驗證結果標記為「有效」。
- 對於 **XYZ-9999** 和 **A12B34C** :
 - 回傳對應錯誤信息並記錄失敗原因於日誌中。

案例 2：資料儲存測試

- 測試目標：確認驗證結果能正確儲存於資料庫。
- 輸入條件：
 - 車牌號碼：ABC-1234
 - 驗證結果：有效
 - 車主姓名：張三
 - 地址：台北市中正區
- 預期結果：
 - 資料庫新增一筆記錄至 **VehicleInfo** 表。

2.4 測試範圍

涵蓋合法性驗證、格式檢查、異常處理與數據儲存。

2.5 測試環境

伺服器：本地端Express.js；資料庫：MySQL；工具：Postman 或 Python 腳本。

2.6 測試總結

透過上述測試案例，可以確保車籍驗證子系統能正確處理車牌資料，返回準確的車主信息，並對異常情況進行有效處理。同時，驗證結果能被正確記錄於資料庫中，支援後續流程。完整內容已根據需求進行整理並提升可讀性。如需補充其他模組或細節，可進一步提供需求！

3. AI 辨識子系統測試腳本

1. 測試目標

確保 AI 辨識子系統能正確處理影像數據，準確辨識車牌號碼、車型與車輛顏色，同時處理失敗的情境並記錄相關信息，並根據錯誤碼機制實現更精細的錯誤管理。

2. 測試項目

1. 影像數據處理測試

2. 車牌號碼辨識測試
 3. 車型辨識測試
 4. 車輛顏色辨識測試
 5. 辨識失敗處理測試
 6. 資料儲存測試
 7. 錯誤碼機制測試
-

3. 測試案例

案例 1：影像數據處理測試

- 測試目標：確認系統能正確處理並標準化輸入的影像數據。
 - 輸入條件：
 - 違規影像：`image001.jpg`（解析度：1920x1080，格式：JPEG）。
 - 測試步驟：
 1. 啟動 AI 辨識子系統。
 2. 傳入影像數據 `image001.jpg`。
 3. 檢查系統日誌中處理的影像數據格式。
 - 預期結果：
 - 系統將影像數據標準化（如調整解析度為 1280x720）。
 - 系統記錄處理後的影像數據（格式、大小等）。
-

案例 2：車牌號碼辨識測試

- 測試目標：確認 AI 能準確辨識影像中的車牌號碼。
- 輸入條件：
 - 違規影像：車牌清晰可見，號碼為 `ABC-1234`。
- 測試步驟：
 1. 傳入影像 `image002.jpg`。
 2. 系統執行車牌辨識。
 3. 檢查辨識結果。
- 預期結果：

- 系統成功辨識車牌號碼為 ABC-1234 。
 - 辨識結果儲存至 AIRecognition 表，標記為「成功」。
-

案例 3：辨識失敗處理測試

- 測試目標：確認 AI 無法辨識影像時能正確記錄錯誤資訊，並傳遞至人工審核子系統。
 - 輸入條件：
 - 違規影像：影像模糊，無法辨識車牌號碼。
 - 測試步驟：
 1. 傳入影像 image005.jpg 。
 2. 系統執行辨識處理。
 3. 檢查錯誤記錄是否正確。
 - 預期結果：
 - 系統回傳辨識失敗，錯誤類型為「影像模糊」。
 - 記錄錯誤原因至 AIRecognition 表。
 - 將影像數據傳遞至人工審核子系統。
-

案例 4：資料儲存測試

- 測試目標：確認 AI 辨識結果能正確儲存至資料庫。
 - 輸入條件：
 - 車牌號碼：ABC-1234
 - 辨識狀態：成功。
 - 測試步驟：
 1. 完成 AI 辨識後，檢查資料庫 AIRecognition 表。
 - 預期結果：
 - AIRecognition 表新增一筆記錄，包含以下內容：
 - 車牌號碼：ABC-1234
 - 錯誤類型：無
-

案例 5：錯誤碼機制測試

- **測試目標：**驗證系統能正確根據辨識結果生成錯誤碼。
 - **輸入條件：**
 - 車牌號碼辨識信心度：低於 90%。
 - 圖片連結不可用。
 - 車速超速：超過速限 30 km/h。
 - **測試步驟：**
 1. 傳入影像與相關違規數據至系統。
 2. 系統執行辨識與錯誤檢查。
 3. 檢查錯誤碼記錄是否正確。
 - **預期結果：**
 - 系統針對信心度低於 84% 的辨識結果標記錯誤碼並記錄至 **AIRecognition** 表。
 - 若圖片連結不可用，標記錯誤碼 **404**。
 - 若超速超過 30 km/h，生成錯誤碼並附加「設備問題」標註。
-

4. 測試範圍

- 涵蓋影像數據的標準化處理、車輛特徵（車牌號碼、車型、顏色）辨識、辨識失敗處理、錯誤碼生成與數據儲存。
 - 涉及的資料表：
 - **AIRecognition**：儲存辨識結果、狀態與錯誤資訊。
 - **ProcessingLog**：記錄辨識失敗或異常情況。
-

5. 測試環境

- 測試伺服器：本地端Express.js
 - AI 模型：Gemini Flash 2.0 Exp. API 辨別
 - 資料庫：MySQL 8.0
 - 模擬工具：影像生成工具或測試資料集。
-

6. 測試總結

上述測試案例涵蓋了 AI 辨識子系統的核心功能，確保影像數據能正確處理，在可以正確辨識車牌(84%)，並能有效記錄辨識失敗與錯誤碼相關信息。

4. 人工辨識子系統測試腳本

1. 測試目標

確保人工辨識子系統能接收 AI 辨識失敗或低信心數據，並能進行人工審核操作，包括車牌資料的確認、記錄審核歷程，並更新案件處理狀態。此外，測試系統與全國舉發案件處理檔的互動，驗證系統對員工操作的記錄與流暢性。

2. 測試項目

1. 數據接收與人工審核測試
 2. 數據更新與異常處理測試
 3. 審核歷程記錄測試
-

3. 測試案例

案例 1：數據接收與人工審核測試

- 測試目標：
 - 確認子系統能正確接收來自 AI 辨識子系統與全國舉發案件處理檔的數據。
 - 確認人工審核能正確更新車牌資料，並處理 AI 辨識失敗的情況。
- 輸入條件：
 1. AI 辨識失敗數據：
 - 車牌號碼：未辨識。
 - 錯誤類型：影像模糊。
 - 錯誤碼：404。
 2. 人工確認數據：
 - 車牌號碼：ABC-1234。
- 測試步驟：
 1. 啟動人工辨識子系統。
 2. 從 AI 辨識子系統與全國舉發案件處理檔傳入數據。
 3. 模擬人工操作更新車牌號碼為 ABC-1234。
 4. 檢查更新後數據是否正確儲存。

- **預期結果：**
 - 系統成功接收數據並顯示待審核案件。
 - 人工確認後，數據更新為：
 - 車牌號碼：ABC-1234。
 - 更新結果儲存至 `ProcessingLog` 和 `EventBasicInfo`。
-

案例 2：數據更新與異常處理測試

- **測試目標：**確認人工審核完成後，數據能正確傳遞至罰單生成子系統，並能處理錯誤操作情況。
 - **輸入條件：**
 1. 經人工確認的車牌號碼：XYZ-5678。
 2. 提交的數據：車牌號碼為空（未填寫）。
 - **測試步驟：**
 1. 完成案件人工審核並傳遞至罰單生成模組。
 2. 模擬提交不完整數據，檢查系統的錯誤提示。
 - **預期結果：**
 - 對於完整數據：
 - 系統顯示審核狀態為「已完成」。
 - 審核結果正確傳遞至罰單生成子系統。
 - `TicketInfo` 表新增一筆對應罰單記錄。
 - 對於不完整數據：
 - 系統提示「車牌號碼為必填項」。
 - 審核操作未提交，狀態維持為「待審核」。
-

案例 3：審核歷程記錄測試

- **測試目標：**確認每次人工審核的處理歷程能正確記錄，包括審核人員、時間和修改內容。
- **輸入條件：**
 - 人工審核操作：
 - 車牌號碼由 `未辨識` 更新為 `DEF-4321`。

- 操作人員：員工 A。
 - 測試步驟：
 1. 完成案件人工審核。
 2. 查詢 `ProcessingLog` 表中的記錄。
 - 預期結果：
 - `ProcessingLog` 表新增一筆記錄，內容包括：
 - 修改前車牌號碼：未辨識。
 - 修改後車牌號碼：DEF-4321。
 - 操作人員：員工 A。
 - 審核時間：記錄正確的時間戳。
-

4. 測試範圍

- 涵蓋數據接收、人工審核操作、數據更新、錯誤處理與審核歷程記錄。
 - 涉及的資料表：
 - `ProcessingLog`：儲存人工審核歷程。
 - `EventBasicInfo`：更新審核後的違規案件數據。
 - `TicketInfo`：傳遞至罰單生成子系統後生成罰單記錄。
-

5. 測試環境

- 測試伺服器：本地端Express.js
 - 前端模擬：人工審核介面（React + RESTful API）。
 - 資料庫：MySQL 8.0。
 - 測試工具：Postman 或 Selenium 自動化腳本。
-

6. 測試總結

上述測試案例確保人工辨識子系統能正確處理 AI 辨識失敗的數據，準確更新車輛信息，並能有效記錄審核歷程與操作 Log，支援後續的罰單生成與案件管理。

5. 罰單生成子系統測試腳本

1. 測試目標

確保罰單生成子系統能正確接收處理後的違規案件數據，生成罰單，通知違規民眾，並記錄處理狀態與相關信息。

2. 測試項目

1. 罰單生成與通知測試
 2. 數據儲存與異常處理測試
 3. 處理歷程記錄測試
-

3. 測試案例

案例 1：罰單生成與通知測試

- **測試目標：**確認系統能正確生成罰單，包含所有必需數據，並通知車主。
- **輸入條件：**
 - 經人工審核完成的違規案件數據：
 - 車牌號碼：ABC-1234
 - 違規時間：2024-12-22 14:30
 - 違規地點：台北市信義區
 - 道路速限：60 km/h
 - 車輛時速：85 km/h
 - 罰金金額：3000 元
 - 車主信息：
 - 姓名：張三
 - 地址：台北市信義區中正路100號
 - 聯絡方式：0987-123-456
- **測試步驟：**
 1. 啟動罰單生成子系統。
 2. 傳入以上違規案件數據。
 3. 檢查生成的罰單數據。
 4. 模擬通知系統發送罰單信息至車主。
 5. 檢查通知狀態。

- **預期結果：**

- 系統生成一張罰單，包含以下信息：
 - 車牌號碼：ABC-1234
 - 違規時間與地點：2024-12-22 14:30，台北市信義區
 - 道路速限與車輛時速：60 km/h，85 km/h
 - 罰金金額：3000 元
 - 罰單編號：TICKET-001
- 通知內容如下：
 - 「尊敬的車主張三，您的車輛 (ABC-1234) 在台北市信義區因超速被開罰單，金額為 3000 元。」
- 罰單數據正確儲存至 `TicketInfo` 表。
- 通知狀態記錄於 `TicketInfo` 表，標記為「已通知」。

案例 2：數據儲存與異常處理測試

- **測試目標：**確認所有罰單生成數據能正確儲存至資料庫，並處理生成過程中的異常情況（如缺失關鍵數據）。
- **輸入條件：**
 1. 正常案件數據：
 - 車牌號碼：DEF-5678
 - 違規時間：2024-12-23 10:00
 - 違規地點：新北市板橋區
 - 罰金金額：5000 元
 2. 不完整數據：
 - 車主信息缺失：
 - 車牌號碼：GHI-7890
 - 違規時間：2024-12-24 08:30
 - 車主姓名：未提供
 - 罰金金額：4000 元
- **測試步驟：**
 1. 傳入正常案件數據至系統生成罰單。

2. 檢查資料庫 `TicketInfo` 表。
 3. 傳入不完整的案件數據至系統。
 4. 檢查系統錯誤提示與處理結果。
- 預期結果：
 - 正常數據：
 - `TicketInfo` 表新增一筆記錄，包含以下內容：
 - 車牌號碼：DEF-5678
 - 違規時間與地點：2024-12-23 10:00，新北市板橋區
 - 罰金金額：5000 元
 - 通知狀態：未通知
 - 不完整數據：
 - 系統提示錯誤：「缺少車主姓名，無法生成罰單」。
 - 案件數據未寫入 `TicketInfo` 表。
-

案例 3：處理歷程記錄測試

- 測試目標：確認每張罰單的生成與通知歷程能正確記錄。
- 輸入條件：
 - 車牌號碼：JKL-1122
 - 違規時間：2024-12-25 16:00
 - 罰金金額：2000 元
 - 操作人員：員工 A
- 測試步驟：
 1. 完成罰單生成與通知流程。
 2. 查詢資料庫 `ProcessingLog` 表。
- 預期結果：
 - `ProcessingLog` 表新增一筆記錄，內容包括：
 - 罰單編號：TICKET-002
 - 操作人員：員工 A
 - 操作時間：記錄正確的時間戳。

- 操作類型：罰單生成與通知。
 - 操作狀態：成功。
-

4. 測試範圍

- 涵蓋罰單生成、車主通知、數據儲存、異常處理與處理歷程記錄。
 - 涉及的資料表：
 - `TicketInfo`：儲存罰單數據與通知狀態。
 - `ProcessingLog`：記錄操作歷程與處理狀態。
-

5. 測試環境

- 測試伺服器：本地端Express.js
 - 資料庫：MySQL 8.0
 - 通知模擬工具：SMS API 或 Email 模擬工具。
-

6. 測試總結

上述測試案例涵蓋了罰單生成子系統的核心功能，確保罰單能正確生成並通知車主，並能處理異常情況，同時完整記錄處理歷程。如需補充其他特殊場景測試，請進一步提供需求！

6. 全國違規事件管理儀表板子系統測試腳本

1. 測試目標

確保儀表板子系統能準確整合全國違規事件數據，提供統計分析功能，並能正確生成視覺化報表供交通管理單位查詢。

2. 測試項目

1. 視覺化報表生成測試

3. 測試案例

案例 1：視覺化報表生成測試

- 測試目標：確認儀表板能生成正確的視覺化報表（如柱狀圖與圓餅圖）。
- 輸入條件：

- 統計數據：
 - 台北市資料大平台車禍統計資料：80%
 - 違規處理子系統之車禍資料：20%
 - 測試步驟：
 1. 啟動儀表板子系統。
 2. 選擇生成違規類型分布的圓餅圖。
 3. 檢查圖表是否準確顯示統計結果。
 - 預期結果：
 - 儀表板生成的圓餅圖顯示：
 - 台北市資料大平台車禍統計資料：80%
 - 違規處理子系統：20%
 - 圖表正確呈現顏色與標籤。
-

4. 測試總結

透過上述測試案例，可以確保全國違規事件管理儀表板子系統能準確整合數據，支持統計分析與視覺化展示，同時具備強大的篩選與導出功能，滿足交通管理單位的需求。

三、開發測試過程

階段一：準備與環境搭建

1. 資料準備：
 - 將測試計劃和DFD圖上傳至ChatGPT的專案空間，作為開發基礎資料。
 - 向AI提出需求，請求開發違規處理資料子系統，並要求AI提供詳細的環境設置指導。
2. 環境搭建：
 - AI詳細指導了工具選擇、程序安裝以及文件夾結構設置，包括：
 - 所需工具和軟體的安裝步驟。
 - 文件和資源的存放位置。
 - 新建必要的文件夾結構。

- 根據AI的指導，逐步完成環境配置，並在IntelliJ中成功搭建Node.js的前後端環境。
- 過程中遇到一些問題，通過調試解決，整個環境搭建耗時約一下午。

階段二：程式碼結構調整

1. 前後端程式碼整合：

- 開發過程中發現前端（React）與後端（Express）的資料夾程式碼分離，不利於協作開發。
- 將前後端代碼合併至同一文件夾，便於統一管理與開發。

2. 改進打包流程：

- 先運行前端React應用並進行打包，再將打包好的文件交由後端Express服務器處理。
- 通過這種方式實現前後端聯動，提升開發效率。

階段三：基本「違規處理子系統」框架搭建與功能開發

1. 生成程式碼框架：

- 向AI給違規處理子系統測試計畫的內容，希望生成基本的違規處理子系統程式碼框架。
- 根據AI指導新增必要文件，在指定文件夾中創建新文件並編輯相應內容。
- 按照指示一步步完成子系統基礎框架的搭建。

2. 功能開發與調試：

- 在開發過程中遇到多次bug，例如變量名稱遺漏或必要資源包未導入等問題。
- 通過逐步調試解決這些問題，確保基礎功能正常運行。

階段四：AI辨識、車籍確認、人工辨識子系統，切換至Tabnine使用

1. AI辨識子系統開發：

- 基本框架完成後，開始進行AI辨識子系統的開發。
- 最初使用ChatGPT進行一般性功能開發，但隨著專案複雜度增加，ChatGPT逐漸不敷使用。

2. 切換至Tabnine：

- 使用IntelliJ中的Tabnine進程式碼撰寫，相較於ChatGPT，其程式碼品質更高且更適合程式開發。

- 一樣給Tabnine關於AI辨識、車籍確認、人工辨識子系統的測試計畫內容，分項給AI產出程式碼。
 - 但Tabnine存在以下問題：
 - 忘記變量名稱或遺漏必要資源包導入，導致bug頻繁出現。
 - 人工需花費大量時間修復錯誤，約有50%的幾率出現報錯。
-

階段五：罰單生成子系統與切換至Vscode Copilot

1. 專案複雜度提升：

- 隨著專案進一步擴展，到了最後罰單生成子系統，Tabnine逐漸無法滿足需求。
- Tabnine對話式協作效率降低，需要人工頻繁複製貼上程式碼，不利於快速迭代。

2. 切換至Copilot：

- 開始使用VS Code中的Copilot進行AI協作編程，其優勢包括：
 - 能記住檔案之間的關係，自動處理相關檔案編輯。
 - 減少語法和資源導入錯誤，提高複雜系統連接處理效率。
 - 無需人工手動操作，只需確認AI產出的檔案是否符合需求。

3. 功能新增與調試：

- 雖然Copilot在語法和導入方面表現穩定，但其輸出質量不一致，有時偏離需求。
 - 需要花費更多時間調整AI輸出的代碼，以確保其符合預期功能，所以更多時間是在閱讀AI產出的程式碼，進而Debug、調適Prompt，等待AI給予程式碼，個人沒有實際寫到多少code。
-

總結：人工介入的重要性

- AI工具在單一功能實現上表現優秀，但在整體系統開發中仍需人工參與來補足不足之處。
 - AI擅長穩健且單一的功能開發，但從0到1構建整個系統，以及從1到N進行擴展時，都需要人工介入以確保穩定性和準確性，這就需要開發人員對於專案和應用的技術有基本的認識。
 - AI在系統開發上給的Code是可以運行的，但品質就是中規中矩的，好的Code仍然須由好的程式設計師寫。
-

四、檢討分析報告

個人能力檢討

1. 技術能力不足

- 雖然期末報告時發表時出現了一些狀況，就是發表時筆記型電腦無法連接HDMI，導致實際演示效果不佳，只能播放影片，效果和其他組相比有侷限性，但是發表時還算順利。
- 然而，與其他組別相比較後，自覺我們開發的內容表現較平庸，雖投入大量心力，但仍感到能力有限，尤其在處理後端系統的複雜性問題時顯得力不從心，或許這是大多由AI開發的侷限性所在，AI開發比較多給予中規中矩的程式碼。
- 在使用AI開發過程中，出現許多難以修復的Bug，耗費大量時間。由於大多程式碼由AI生成，雖嘗試理解，但對於解決多系統相互連結的Bug缺乏系統性方法，去修正這些錯誤。

2. 學習與提升空間

- 這讓我們了解到，面對技術瓶頸時，自身知識儲備是較為不足的，尤其在後端系統的錯誤排查與修復方面，不可全由AI開發。所以，要成為一位好的專業軟體工程師，系統性的學習背景知識和技術是很必要的，我們後續需要進一步加強對Bug修復流程及後端架構的學習，特別是多系統整合的處理。

團隊合作檢討

1. 分工與執行效率，與專案難度過高

- 團隊內大部分工作由組長一人負責，以及CTO處理儀表板事項，而其他成員在面對高難度任務時表現出能力不足。並且任務分配後常出現長時間未完成或完成品質不佳的情況，導致進度延誤。
- 這讓我們反思到，很大原因在於此專案難度，超出團隊整體能力範圍，導致成員無法快速適應和在短時間內（約1個月內）完成任務。所以，專案規劃充分考慮團隊能力與專案需求之間的匹配性還是很重要的，有多少能力做多少事。

2. 團隊凝聚力不足

- 因為專案前期大多為組長一人作業，如何接替工作組長也沒有詳細的說明，導致後續組長下放工作時，組員們比較難以接替工作內容，或內容太多組員吸收不了，而又回到給組長處理工作。而後團隊內部分化趨勢明顯，各自為政的情況增加，部分成員甚至擅自決定計劃外事項，導致組長變得難以管理。關於組織提效方面，我們缺乏有效的溝通和協作機制，使得團隊整體效率受到影響。

改進策略

1. 針對個人能力

- 強化技術基礎：專注於後端開發相關技能的學習，包括錯誤排查、系統整合及架構設計。
- 提升問題解決能力：學習更多關於Debugging工具和方法，並參考相關案例研究來提高實戰經驗。

2. 針對團隊合作

- 明確分工與責任：根據成員能力合理分配任務，同時設置階段性檢查點以確保進度。
- 加強溝通與協作：定期舉行會議討論進展及問題，確保所有成員了解項目全貌並參與決策。
- 增強團隊凝聚力：舉辦團建活動或設置共同目標，提高成員間的信任與合作意願。

3. 針對專案規劃

- 調整專案範圍：根據團隊能力適當縮減專案規模或降低技術難度。
- 設置學習階段：在專案初期安排技術培訓或簡單模組練習，以提升成員技能水平。
- 引入外部資源：考慮尋求外部指導或使用更成熟的工具來降低開發難度。